

# TASK 0

## Python Fundamentals, Data Analysis & Git

A progressive introduction to Python, NumPy, Pandas, Matplotlib, and a clean GitHub submission workflow.

### Submission Guidelines

Submit this task through a **public GitHub repository**. Your repository should contain your source code, the provided dataset, generated output files where requested, and a README with instructions for running your work.

### Recommended repository structure

```
task-0/  
  README.md  
  q1.py  
  q2.py  
  q3.py  
  q4.py  
  q5.py  
  q6.py  
  data/  
  student_performance.csv  
  processed_student_performance.csv  
  plots/  
    final_scores.png  
    study_vs_score.png  
    score_distribution.png  
    custom_plot.png
```

### Git & GitHub requirements

- Create a Git repository for the task and push it to GitHub.
- Make meaningful commits as you progress. Do not upload the entire completed task as a single first commit.
- Use clear commit messages, for example: Complete Q1 list analyzer or Add Pandas CSV analysis.
- Include a README.md containing your name, a short description of the task, setup instructions, and commands needed to run the solutions.

- Do not commit virtual environments, cache folders, or unnecessary generated files. Using a `.gitignore` is recommended.
- Before submitting, clone or inspect your repository and verify that another person can follow the README and run your code.

**Submission:** Submit the URL of your GitHub repository. The repository itself is the final submission.

**General rule:** Unless a question explicitly allows otherwise, write readable code, use meaningful variable names, and avoid copying solutions without understanding them.

## Q1. List Analyzer

Take **N** integers as input and store them in a list. Without using `max()`, `min()`, `sum()`, `sort()`, or `sorted()`, find and print the following:

1. The largest element.
2. The smallest element.
3. The sum of all elements.
4. The number of even elements.
5. The number of odd elements.
6. The list in reverse order.

### Example Input

```
6
4 7 2 9 6 3
```

### Example Output

```
Largest: 9
Smallest: 2
Sum: 31
Even count: 3
Odd count: 3
Reversed: 3 6 9 2 7 4
```

**Focus:** lists, loops, conditions, indexing.

## Q2. Lists, Functions and .copy()

Write a function with the following signature:

```
def process_list(numbers):  
    # Your code
```

The function takes a list of integers as input and must return a **new list** after performing these operations:

1. Create a copy of the input list.
2. Remove all negative numbers from the copied list.
3. Append 0 to the list.
4. Sort the list in ascending order.
5. Return the modified list.

The **original list must remain unchanged**.

### Example

```
original = [5, -2, 8, -1, 3]  
  
result = process_list(original)  
  
print("Original:", original)  
print("Result:", result)
```

### Expected Output

```
Original: [5, -2, 8, -1, 3]  
Result: [0, 3, 5, 8]
```

Use and understand list methods such as `.copy()`, `.append()`, `.remove()`, and `.sort()`.

Before solving, observe the difference between:

```
list2 = list1  
  
# and  
  
list2 = list1.copy()
```

The returned list may be modified, but the list passed to the function must not change.

**Focus:** functions, arguments, return values, list methods, mutable objects, and copying lists.

## Q3. Prime Numbers Using for-else

Write a function:

```
def is_prime(n):  
    # Your code
```

The function should return True if n is prime and False otherwise. You must use Python's for-else syntax.

### Example

```
print(is_prime(7))  
print(is_prime(12))
```

### Output

```
True  
False
```

Your solution should use this general control-flow structure:

```
for ...:  
    if ...:  
        break  
else:  
    ...
```

Then use your function to print all prime numbers from **2** to **N**.

### Example Input

```
20
```

### Example Output

```
2 3 5 7 11 13 17 19
```

Add a comment in your code explaining: **When does the else block associated with a for loop execute?**

**Focus:** functions, nested loops, break, for-else, and prime checking.

## Q4–Q6. Linked Data Analysis Project

Questions 4, 5, and 6 form one connected mini-project. You will move from numerical arrays to tabular data and finally to visualizations.

**Pipeline:** NumPy computation → Pandas CSV analysis → Matplotlib visualization

### Dataset

A file named `student_performance.csv` is provided with this task. It contains the following columns:

| Column         | Meaning                   |
|----------------|---------------------------|
| Student        | Student name              |
| Hours_Studied  | Average study hours       |
| Attendance     | Attendance percentage     |
| Previous_Score | Previous assessment score |
| Final_Score    | Final assessment score    |

Keep the original dataset unchanged. Any processed version should be saved as a separate file.

## Q4. NumPy Basics

Using **NumPy**, create arrays containing the values for:

- Hours studied
- Attendance
- Previous scores
- Final scores

For this question, the values may be provided directly in your code. Do **not** read the CSV file yet.

### Perform the following operations:

1. Print the shape and data type of each array.
2. Find the mean final score.
3. Find the maximum and minimum final score.
4. Find the standard deviation of final scores.
5. Add 5 bonus marks to every final score using NumPy array operations.
6. Create a Boolean array showing which students scored at least 75.
7. Use Boolean indexing to print only the scores greater than or equal to 75.

### Example array

```
scores = np.array([70, 91, 58, 87, 76])
```

**Restriction:** Use NumPy operations for the calculations above rather than manually looping through the arrays.

**Focus:** `np.array`, `shape`, `dtype`, vectorized operations, `mean`, `min`, `max`, `std`, and Boolean indexing.

## Q5. Pandas and CSV Analysis

Now read the complete dataset from `student_performance.csv` using **Pandas**.

### Perform the following tasks:

1. Load the CSV into a DataFrame.
2. Print the first five rows.
3. Print the number of rows and columns.
4. Display the column names.
5. Check whether the dataset contains missing values.
6. Calculate the average `Final_Score`.
7. Find the student with the highest `Final_Score`.
8. Create a new column: `Improvement = Final_Score - Previous_Score`.
9. Display only students with attendance greater than or equal to 80.
10. Sort the DataFrame by `Final_Score` in descending order.
11. Save the processed DataFrame as `processed_student_performance.csv`.

**Focus:** `pd.read_csv()`, DataFrames, columns, filtering, creating columns, sorting, and CSV input/output.



## Q6. Visualizing the Data with Matplotlib

Using the processed data from **Q5**, create the following visualizations with **Matplotlib**:

1. **Bar chart:** Student names vs final scores.
2. **Scatter plot:** Hours studied vs final score.
3. **Histogram:** Distribution of final scores.
4. One additional graph of your choice that shows an interesting relationship or pattern in the dataset.

### Every graph must contain:

- A clear title.
- Labels for the x-axis and y-axis.
- Appropriate formatting so that the graph is readable.

### Save each graph as an image:

```
final_scores.png  
study_vs_score.png  
score_distribution.png  
custom_plot.png
```

**Focus:** `plt.bar()`, `plt.scatter()`, `plt.hist()`, labels, titles, readability, and saving figures.

## Final Submission Checklist

- All six questions are completed and committed.
- The provided CSV is included in the repository.
- The processed CSV generated in Q5 is included.
- All four plot images generated in Q6 are included.
- README instructions are sufficient for another person to install dependencies and run the work.
- The GitHub repository URL is submitted.

Build it, commit it, push it. Your Git history should tell the story of your progress, not merely reveal the final snapshot.